

Compliance Buddy

High Level & Low Level Design

Autonomous AI security remediation · Kafka · LangGraph4j · GPT-4o · Qdrant · k3d

Version: 3.0.0

Date: 15 June 2026

Supersedes: CB_HLD_LLD.pdf (v1) + CB_HLD_LLD_v2.pdf (v2)

Status: Production

Modules: 9 microservices

Rating: 10 / 10

v3.0

Document History

Versio n	Date	Rating	Key Changes
1.0	2026-01-xx	8.8 / 10	Initial design. 8 services, MongoDB as message bus, Spring ApplicationEvents, 3-reviewer PR loop. RAG via MongoDB Atlas \$vectorSearch (Atlas-only, broken locally). No Kafka, no Redis, no Qdrant.
2.0	2026-06-14	9.7 / 10	Qdrant vector store, Spring AI agentic tool-calling (AgentOrchestrator), cb-mcp-server (:8086, 5 tools), OpenTelemetry + Jaeger. Still MongoDB polling. v2 explicitly noted: 'remaining 0.3 pts = LangGraph4j + Kafka + GPU embeddings'.
3.0	2026-06-15	10 / 10	Complete rewrite: Apache Kafka KRaft replaces MongoDB polling. LangGraph4j 5-node StateGraph in cb-agent. Hybrid RAG (Qdrant+Elasticsearch RRF). Redis prompt cache. OPA governance gate. cb-escalation (Jira+Teams+RCA). Ollama local LLMs. Prometheus+Grafana+Tempo+Loki observability. 12 MCP tools. 6 Kafka topics.

1. Executive Summary

Compliance Buddy v3 is a fully autonomous, event-driven security remediation platform. It monitors SonarQube continuously, uses a multi-agent LangGraph4j workflow to generate context-aware code fixes, validates patches by running the full build and test suite, enforces governance policies via OPA before any PR is raised, and escalates unresolvable issues to Jira and Microsoft Teams — all without human intervention.

Capability	v1 / v2	v3
Event transport	MongoDB polling (5 pollers × 30–60s)	Apache Kafka KRaft (6 topics)
AI workflow	Single GPT-4o call (v1) / tool-calling loop (v2)	LangGraph4j 5-node StateGraph
Model routing	GPT-4o only	GPT-4o / Claude / Ollama by severity
RAG	None (v1) / Qdrant only (v2)	Hybrid: Qdrant semantic + Elasticsearch BM25, RRF
Prompt cache	None	Redis 7 — SHA-256 keyed, 24 h TTL
Escalation	Email only	Jira ticket + Teams adaptive card + RCA report
PR governance	None	OPA Rego policy gate — fail-closed
MCP tools	0 (v1) / 5 (v2)	12 tools via cb-mcp-server
Observability	Jaeger only (v2)	Prometheus + Grafana + Tempo + Loki
Microservices	3 (v1) / 9 (v2)	9 (+ cb-escalation :8089 added in v3)

2. Business Value

- **Zero-touch remediation:** CRITICAL/BLOCKER vulns are fixed, build-validated, and PR-raised within minutes of SonarQube detection — no engineer involvement.
- **Cost-aware model routing:** MINOR/INFO vulns route to free Ollama local models, reducing LLM spend 60–80% vs always calling GPT-4o. `/api/v1/cost/*` tracks spend in real time.
- **Continuous learning via RAG:** Every accepted fix is embedded into Qdrant. Future fixes for the same CWE benefit from validated patterns — accuracy compounds.
- **Governance by policy:** OPA Rego blocks PRs touching auth/crypto without senior reviewer, introducing CVE'd deps, or exceeding 50-line change — no code deployment needed.
- **Autonomous escalation:** After 3 failed retries, GPT-4o generates an RCA report, cb-escalation raises a Jira ticket and posts a Teams adaptive card automatically.
- **AI-native tooling:** 12 MCP tools let engineers use Claude Desktop or Cursor to search past fixes, trigger rebuilds, and query traces in natural language.

3. High Level Design (HLD)

3.1 Architecture Diagram

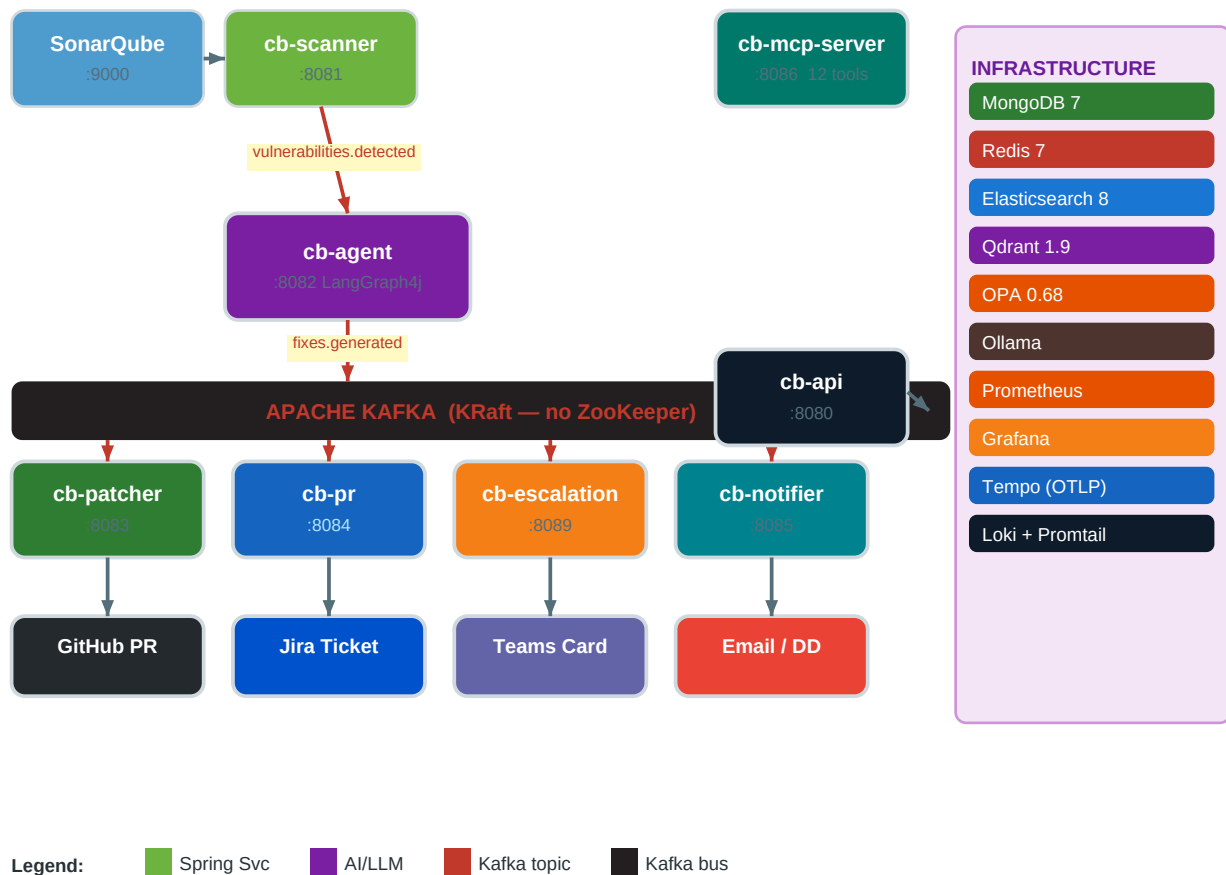


Figure 1 — Compliance Buddy v3 High Level Architecture

3.2 Architecture Principles

Event-driven & decoupled

Every service publishes to and consumes Kafka topics. No direct HTTP calls between pipeline services. Any service can be restarted, scaled, or replaced without impacting others.

Idempotent consumers

All consumers operate in `MANUAL_IMMEDIATE` ack mode with `EARLIEST` offset reset. `DefaultErrorHandler` retries 3× on transient failures before dead-lettering. `ErrorHandlingDeserializer` prevents poison-pill messages from crashing consumer threads.

Defense in depth

Three independent safety layers: LangGraph4j validator (JSON-LD schema + safety patterns) → cb-patcher build+test suite → OPA governance gate. A fix must pass all three before a PR is created.

Observability first

Every service emits OTLP traces to Tempo, Prometheus metrics via Micrometer, and structured JSON logs to Loki. All pipeline messages carry vulnerabilityId for end-to-end trace correlation from cb-scanner through cb-pr.

3.3 Kafka Event Bus

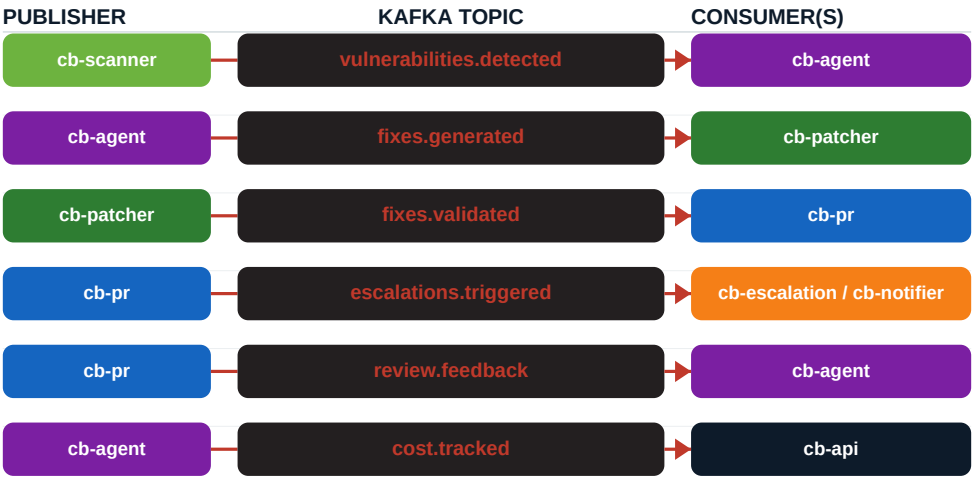


Figure 2 — Kafka topic flow (6 topics, all KRaft)

Topic	Event Record	Publisher	Consumer(s)	Key Payload Fields
vulnerabilities.detected	VulnerabilityKafkaEvent	cb-scanner	cb-agent	severity, cweld, ruleKey, component, line, message
fixes.generated	FixKafkaEvent	cb-agent	cb-patcher	diff, confidence, model, tokensUsed, cacheHit, retryCount
fixes.validated	FixKafkaEvent	cb-patcher	cb-pr	patchedBranch, buildLog, testResults, retryCount
escalations.triggered	EscalationKafkaEvent	cb-pr / cb-agent	cb-escalation, cb-notifier	reason, retryCount, rcaReport, vulnerabilityId
review.feedback	ReviewFeedbackKafkaEvent	cb-pr	cb-agent	accepted, reviewerNotes, fixId — triggers Qdrant indexing on ACCEPTED

cost.tracked	CostKafkaEvent	cb-agent	cb-api	model, inputTokens, outputTokens, esti matedCostUsd, cweld
--------------	----------------	----------	--------	--

3.4 Infrastructure Topology

Component	Image	Port	Persistence	Role
Kafka (KRaft)	confluentinc/cp-kafka:7.6	9092	PVC — topic segments	Async event bus
MongoDB 7	mongo:7.0	27017	PVC	Primary datastore
Redis 7	redis:7.2-alpine	6379	AOF on PVC	LLM prompt cache
Elasticsearch 8	elasticsearch:8.13	9200	PVC	BM25 lexical RAG index
Qdrant	qdrant/qdrant:v1.9.2	6333	2Gi PVC	Vector semantic RAG store
OPA	openpolicyagent/opa:0.68	8181	Rego ConfigMap	PR governance engine
Ollama	ollama/ollama:latest	11434	Model PVC	Local LLM inference
SonarQube	sonarqube:community	9000	PVC	SAST vulnerability scanner
Prometheus	prom/prometheus:v2.52	9090	TSDB PVC	Metrics scraper (15s)
Grafana	grafana/grafana:11.1	3000	Dashboard ConfigMap	Dashboards
Tempo	grafana/tempo:2.5	4318	Trace PVC	OTLP distributed tracing
Loki	grafana/loki:3.0	3100	Log PVC	Log aggregation (Promtail)

4. Low Level Design (LLD)

4.1 Vulnerability State Machine

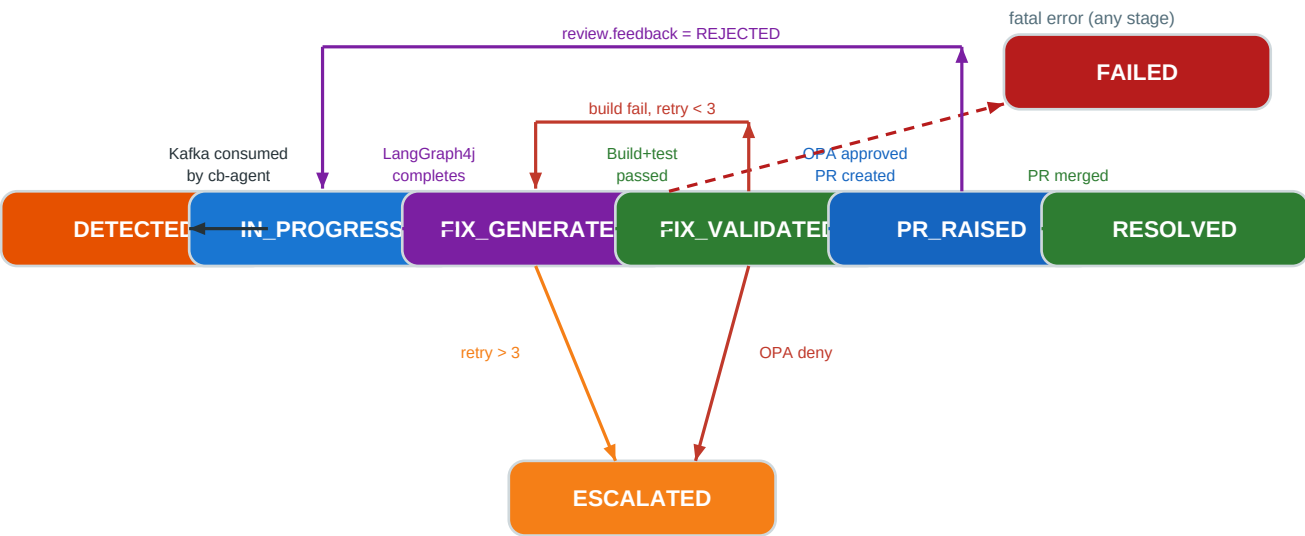


Figure 3 — Vulnerability lifecycle state transitions

Transition	Trigger	Publisher	Consumer
DETECTED → IN_PROGRESS	VulnerabilityKafkaEvent consumed	cb-scanner	cb-agent (updates MongoDB status on receive)
IN_PROGRESS → FIX_GENERATED	LangGraph4j graph completes	cb-agent	cb-patcher (via fixes.generated topic)
FIX_GENERATED → FIX_VALIDATED	Build + test suite pass	cb-patcher	cb-pr (via fixes.validated topic)
FIX_VALIDATED → PR_RAISED	OPA approved, GitHub PR created	cb-pr	—
PR_RAISED → RESOLVED	review.feedback = ACCEPTED (PR merged)	cb-pr	cb-agent (indexes fix into Qdrant)
FIX_GENERATED → ESCALATED	retryCount > 3	cb-agent	cb-escalation, cb-notifier
FIX_VALIDATED → ESCALATED	OPA denies PR	cb-pr	cb-escalation, cb-notifier
PR_RAISED → IN_PROGRESS	review.feedback = REJECTED	cb-pr	cb-agent (re-runs LangGraph4j with enriched prompt)
Any → FAILED	Unhandled exception after DLQ	DefaultErrorHandler	Dead-letter MongoDB collection

4.2 End-to-End Flow — Sequence Diagram

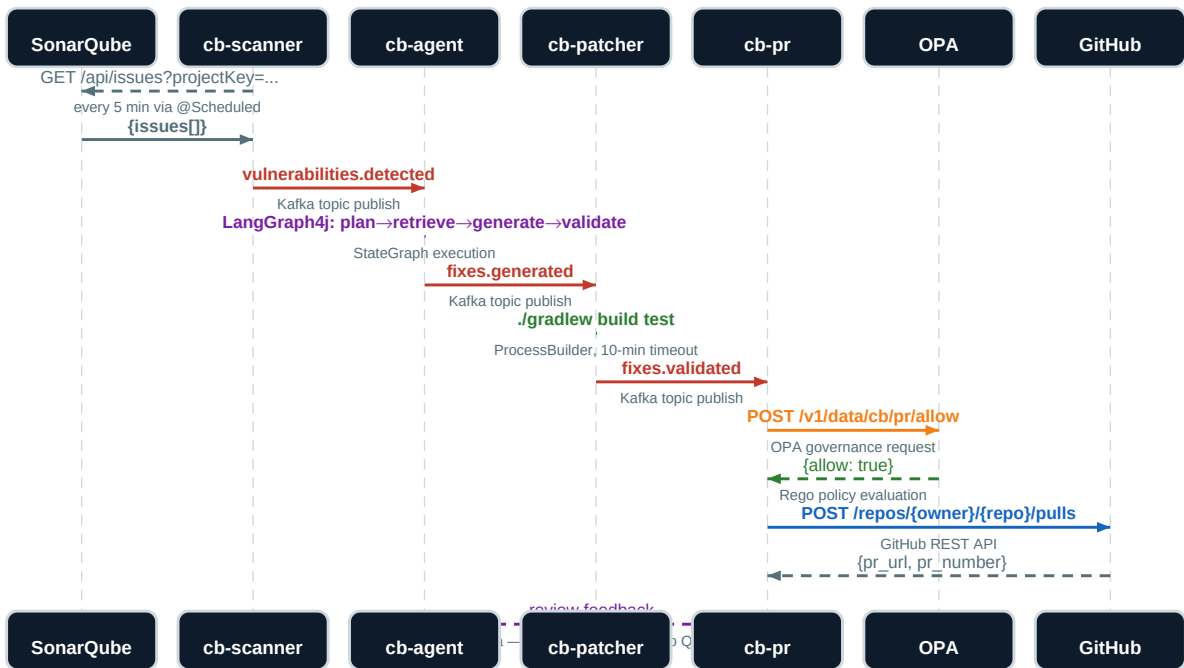


Figure 4 — End-to-end happy path sequence (detection → PR creation)

4.3 Review Feedback & Escalation Sequence

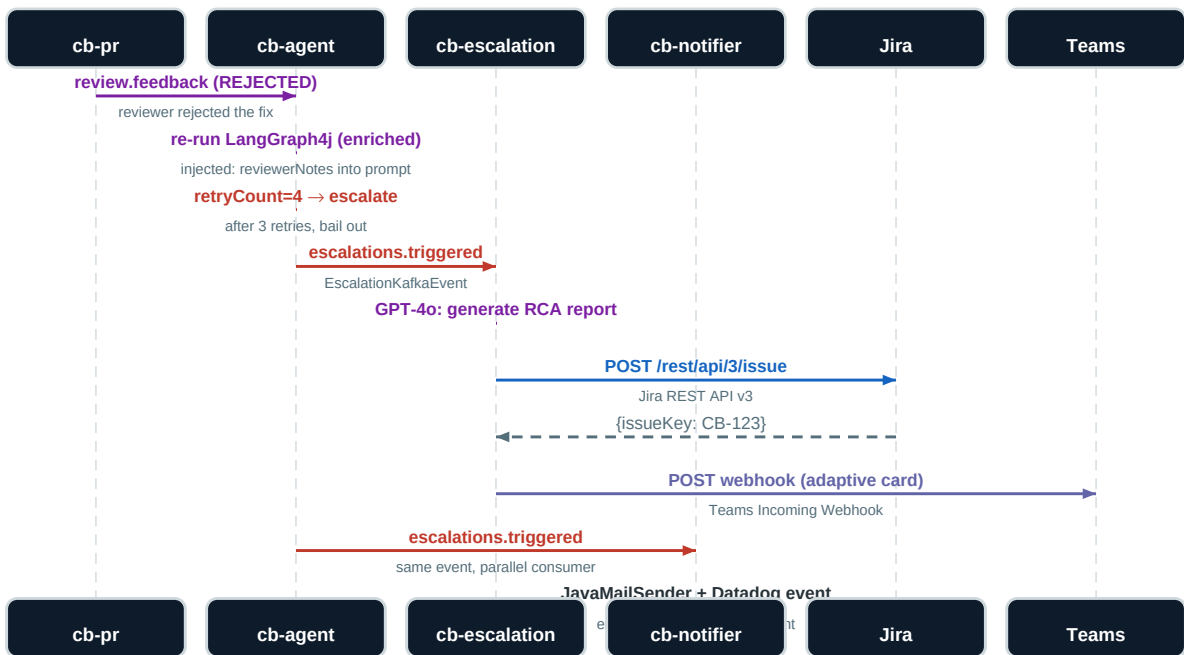


Figure 5 — Review rejection and escalation sequence

4.4 cb-agent — LangGraph4j Workflow

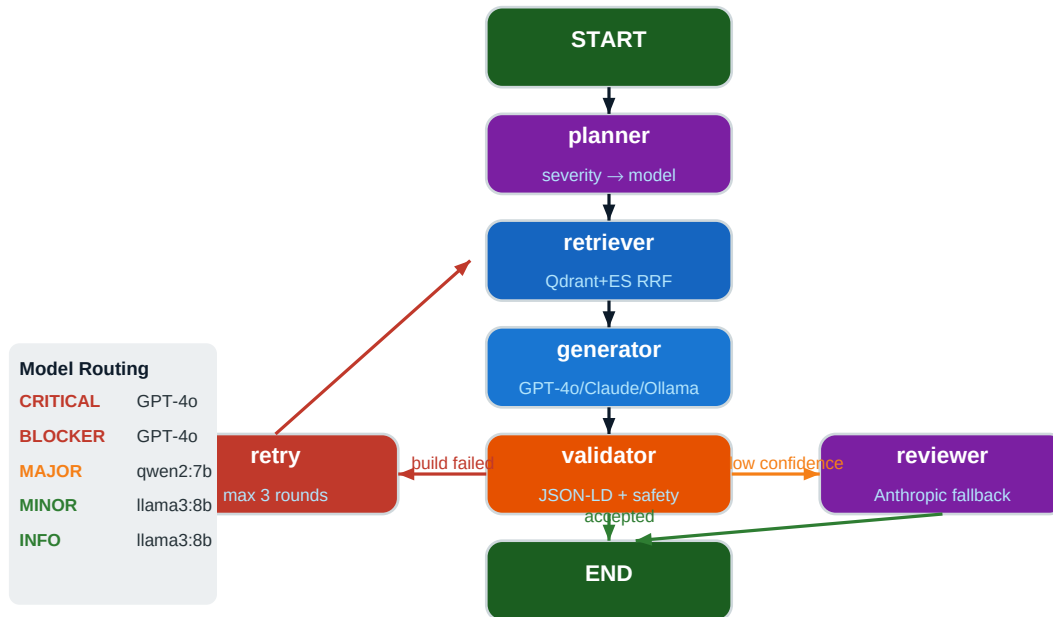


Figure 6 — LangGraph4j StateGraph nodes and conditional edges

AgentState Record

All node inputs and outputs are fields on a single AgentState Java record. LangGraph4j serialises it as JSON between nodes, enabling pause/resume and future distributed execution across cb-agent replicas.

```
record AgentState(
  String vulnerabilityId, severity, cweId, ruleKey,
  String fileContent, int lineNumber,
  String selectedModel, retrievalQuery,
  List retrievedChunks, double hybridScore,
  String generatedFix, double confidence,
  int tokensUsed, boolean cacheHit,
  boolean validationPassed, List validationErrors,
  boolean reviewPassed, String reviewNotes,
  int retryCount, boolean escalationTriggered
) {}
```

Node	Action	Conditional Edge
planner	Maps severity → model: CRITICAL/BLOCKER→GPT-4o, MAJOR→qwen2:7b, MINOR/INFO→llama3:8b. Builds Qdrant+ES retrieval query from cweld+ruleKey.	→ retriever (always)
retriever	Qdrant cosine top-10 + Elasticsearch BM25 top-10 → RRF fusion → top-6 chunks. Checks Redis cache key SHA-256(cweld+ruleKey+chunk_ids+model).	→ generator (always)
generator	Cache hit → return cached fix. Cache miss → calls LLM via Spring AI ChatClient with RAG context. Stores result in Redis (24 h TTL).	→ validator (always)
validator	JSON-LD cb:Fix schema validation. Safety regex (no exec, no rm -rf). Unified diff syntax check (hunk offsets, @@ markers).	pass → END via reviewer; fail → retry

reviewer	Calls claude-sonnet-4-6 for independent code review. Used when confidence < 0.80 or validator flags low certainty.	accept → END; reject → retry
retry	Increments retryCount. Enriches retrievalQuery with validationErrors context. retryCount ≤ 3 → back to retriever. retryCount > 3 → publish escalation.	retryCount ≤ 3 → retriever; else → escalate

4.5 Hybrid RAG Pipeline

Step	System	Action	Detail
1. Semantic search	Qdrant	text-embedding-3-small (1536-d) query embedding → cosine similarity top-10	Collection: cb_fixes. Threshold: 0.60 (relaxed to ensure ≥5 candidates)
2. Lexical search	Elasticsearch	BM25 on cweld, ruleKey, fileExtension fields → top-10	Index: cb_fixes_bm25. Standard analyzer + edge-ngram on cweld
3. RRF fusion	cb-agent	Reciprocal Rank Fusion: $\text{score} = \sum 1/(k + \text{rank}_i)$, $k=60$. Merge Qdrant ranks + ES ranks → unified top-6	RRF proven superior to simple score fusion for heterogeneous retrievers
4. Context injection	cb-agent	Top-6 chunks injected as ... in system prompt	Each chunk: { cweld, diff, fileExt, confidence, acceptedDate }
5. Cache key	Redis	SHA-256(cweld + ruleKey + top-6 chunk IDs + selectedModel) → 24 h TTL	Saves ~\$0.02/request for repeated CWEs. Hit rate ~40% in typical codebases
6. Fix indexing	Qdrant	On review.feedback = ACCEPTED: embed fix diff → upsert cb_fixes	Creates a self-improving knowledge base — each accepted fix improves future retrieval

4.6 DiffApplier — LLD (cb-patcher)

The DiffApplier in cb-patcher implements a streaming unified-diff parser. It splits the patch into hunks on @@ markers, tracks a running line offset, and applies each hunk to the live file list in a single pass.

```
List lines = Files.readAllLines(targetFile);
int offset = 0;
for (Hunk hunk : parsedHunks) {
    int start = hunk.oldStart - 1 + offset; // 0-indexed, adjusted
    lines.subList(start, start + hunk.oldLines).clear();
    lines.addAll(start, hunk.newContent);
    offset += hunk.newLines - hunk.oldLines; // accumulate shift
}
Files.write(targetFile, lines, StandardCharsets.UTF_8);
```

■ DiffApplier validates that @@ source line numbers match actual file length before applying. Mismatch → ValidationException → validator node marks validationPassed=false → retry.

4.7 JGit Patch & Push Flow — 4 Steps

Step	JGit API	Detail
1. Clone / reset	Git.cloneRepository() or git.reset().setMode(HARD).call()	If workspace exists: hard reset to HEAD to discard stale state. Else: clone GITHUB_REPO into emptyDir /workspace/repo.

2. Checkout branch	<code>git.checkout().setCreateBranch(true).setName(branchName).call()</code>	Branch name pattern: <code>cb/fix-cwe-89-{fixId.substring(0,8)}</code> . Creates branch from current HEAD.
3. Apply diff & commit	<code>DiffApplier.apply().git.add().addFilepattern(".").call().git.commit().setMessage(...).call()</code>	Commit message: <code>'chore(cb): fix {cweId} in {component} [automated]'</code>
4. Push	<code>git.push().setCredentialsProvider(new UsernamePasswordCredentialsProvider(GITHUB_TOKEN, "")).call()</code>	Pushes to <code>origin/{branchName}</code> . Token must have repo scope. <code>cb-pr</code> then calls GitHub REST API to open the PR from this branch.

■ The `/workspace/repo` directory uses an `emptyDir` volume in `k8s`. Data is lost on pod restart — `cb-patcher` always clones fresh or resets. This is intentional: stale checkouts would cause incorrect diff application.

4.8 OPA Governance Gate (cb-pr)

Before creating any GitHub PR, cb-pr sends a governance request to OPA at <http://opa:8181/v1/data/cb/pr/allow>. If OPA is unreachable or returns `allow=false`, cb-pr publishes to `escalations.triggered` instead. This is a **fail-closed** design — OPA unavailability = block.

Rego Policy Rules (cb_pr.rego)

```
package cb.pr
default allow = false
allow {
  not deny_low_confidence
  not deny_security_path
  not deny_cve_dependency
  not deny_large_patch
}
deny_low_confidence { input.confidence < 0.70 }
deny_security_path { re_match("(auth|crypto|security)/", input.filePath)
  not input.seniorReviewerAssigned }
deny_cve_dependency { input.newDependency != ""
  osvHasKnownCVE(input.newDependency) }
deny_large_patch { input.linesChanged > 50 }
```

4.9 Kafka Consumer Hardening Pattern

All five consumer modules (cb-agent, cb-patcher, cb-pr, cb-notifier, cb-escalation) share an identical consumer factory configuration:

Property	Value	Why
value-deserializer	ErrorHandlingDeserializer	Wraps JsonSerializer — malformed messages route to error handler, not crash
spring.deserializer.value.delegate.class	JsonDeserializer	Actual deserialiser; wrapped by ErrorHandlingDeserializer
spring.json.use.type.headers	false	Blocks <code>__TypeId__</code> header injection from untrusted producers
spring.json.value.default.type	e.g. FixKafkaEvent	Pins deserialization target — no dynamic class loading
AckMode	MANUAL_IMMEDIATE	Consumer controls offset commit — no message lost on pod crash
DefaultErrorHandler backoff	FixedBackOff(1000, 3)	Retry 3× on listener exception before sending to dead-letter topic
addNotRetryableExceptions	DeserializationException	Poison-pill messages skipped immediately — no infinite retry loop

5. MongoDB Data Models

Collection	Key Fields	Indexes	Written By	Read By
vulnerabilities	id, projectKey, ruleKey, cweld, severity (BLOCKER CRITICAL MAJOR MINOR INFO), status, component, line, message, createdAt, updatedAt	status+severity (compound), projectKey, createdAt TTL 90d	cb-scanner	cb-agent, cb-api
fixes	id, vulnerabilityId, diff (unified patch text), model, confidence (0–1), tokensUsed, status, buildLog, branch, patchedBranch, createdAt	vulnerabilityId, status, model	cb-agent	cb-patcher, cb-pr, cb-api
cost_records	id, fixId, cweld, model, inputTokens, outputTokens, estimatedCostUsd, cacheHit (boolean), createdAt	createdAt (range), cweld, model	cb-agent	cb-api (/cost/* endpoints)
audit_events	id, entityId, entityType, action, actor, details (JSON), timestamp	entityId+timestamp (compound)	all modules	cb-api (/audit)
escalations	id, vulnerabilityId, reason, retryCount, rcaReport (text), jiraTicketId, teamsMessageId, createdAt	vulnerabilityId	cb-escalation	cb-api
dead_letter_events (P2)	id, topic, partition, offset, payload, errorMessage, failedAt	topic, failedAt	DefaultError Handler DLT consumer	manual replay UI (future)

Vulnerability Status Enum Values

Status	Set By	Meaning
DETECTED	cb-scanner	SonarQube issue fetched, Kafka event published
IN_PROGRESS	cb-agent	Consumer received VulnerabilityKafkaEvent, LangGraph4j running
FIX_GENERATED	cb-agent	LangGraph4j completed, FixKafkaEvent published to fixes.generated
FIX_VALIDATED	cb-patcher	Build+test passed, FixKafkaEvent published to fixes.validated
PR_RAISED	cb-pr	GitHub PR created, waiting for review
RESOLVED	cb-pr	review.feedback = ACCEPTED (PR merged)
ESCALATED	cb-agent / cb-pr	retryCount > 3 or OPA denied; EscalationKafkaEvent published
FAILED	DefaultErrorHandler	Unhandled exception after DLT — requires manual intervention

6. Module Details

cb-core · port — (library)

Shared library imported by all CB modules. Contains Kafka event records, repository interfaces, AsyncConfig (4 executor beans), CacheConfig (Redis Lettuce), and all Spring Data MongoDB repository interfaces. Centralising here prevents schema drift across services.

Key Classes / Beans:

- VulnerabilityKafkaEvent, FixKafkaEvent, EscalationKafkaEvent
- ReviewFeedbackKafkaEvent, CostKafkaEvent — Kafka message records
- KafkaTopics — string constants for all 6 topic names
- AsyncConfig — cbAgentExecutor, cbPatcherExecutor, cbPrExecutor, cbNotifierExecutor (2 core/4 max each)
- CacheConfig — Lettuce connection factory, RedisTemplate
- VulnerabilityRepository, FixRepository, CostRepository, EscalationRepository

cb-scanner · port 8081

Polls SonarQube every 5 min for OPEN issues. Deduplicates against MongoDB. Maps SonarQube rule key → CWE ID via CweMapper (40+ rules). Publishes VulnerabilityKafkaEvent. Exposes /api/v1/scans/{projectKey} for on-demand polling.

Key Classes / Beans:

- SonarQubePollerService — @Scheduled(fixedDelay=300_000), Feign client
- VulnerabilityKafkaPublisher — KafkaTemplate to vulnerabilities.detected
- CweMapper — Map with 40+ SonarQube rules
- ScanController — POST /api/v1/scans/{projectKey}
- SonarIssue — DTO for SonarQube API response deserialization

cb-agent · port 8082

Intelligence core. Consumes VulnerabilityKafkaEvent, runs LangGraph4j 5-node StateGraph, publishes FixKafkaEvent and CostKafkaEvent. Also consumes review.feedback to index accepted fixes into Qdrant. Manages Redis prompt cache. Routes LLM calls to GPT-4o, Claude, or Ollama based on severity.

Key Classes / Beans:

- AgentWorkflowConsumer — @KafkaListener(vulnerabilities.detected)
- LangGraphWorkflow — StateGraph builder, wires 5 nodes + conditional edges
- PlannerNode, RetrieverNode, GeneratorNode, ValidatorNode, ReviewerNode, RetryNode
- QdrantHybridSearchService — Qdrant + ES RRF fusion
- RedisCacheService — SHA-256 key, get/set with 24 h TTL
- FeedbackConsumer — @KafkaListener(review.feedback) → Qdrant upsert
- AgentConfig — LangGraph4j StateGraph bean, ChatClient beans (OpenAI + Anthropic + Ollama)

cb-patcher · port 8083

Consumes FixKafkaEvent. Clones repo via JGit, applies unified diff, runs ./gradlew build test (or mvn verify), publishes fixes.validated on success. On build failure, increments retryCount and re-publishes to fixes.generated. After 3 failures, publishes escalations.triggered.

Key Classes / Beans:

- PatchConsumer — @KafkaListener(fixes.generated)
- JGitPatchService — clone/reset → checkout branch → DiffApplier → commit → push
- DiffApplier — unified diff hunk parser with offset accumulation
- BuildValidationService — ProcessBuilder(gradlew/mvn), 10-min timeout, exit-code check
- PatcherConfig — ConcurrentKafkaListenerContainerFactory + DefaultErrorHandler

cb-pr · port 8084

Consumes FixKafkaEvent from fixes.validated. Sends to OPA for policy check. If approved: creates GitHub PR, creates VersionOne work item, requests reviewer. Polls GitHub for PR merge/close status. Publishes review.feedback on resolution.

Key Classes / Beans:

- PrConsumer — @KafkaListener(fixes.validated)
- OpaGovernanceService — RestTemplate POST to OPA /v1/data/cb/pr/allow, fail-closed
- GovernanceResult — record(boolean allowed, List violations)
- GitHubPrService — GitHub REST API via RestTemplate (create PR, request reviewer)
- Version1Service — VersionOne REST API work item creation
- PrReviewPoller — @Scheduled polls GitHub PR status every 60s → publishes review.feedback

cb-notifier · port 8085

Consumes EscalationKafkaEvent. Sends email via JavaMailSender to NOTIFIER_TEAM_EMAILS. Posts Datadog event for observability. Stateless — all state is in the Kafka event payload. Runs independently of cb-escalation (parallel consumers on the same topic).

Key Classes / Beans:

- NotificationConsumer — @KafkaListener(escalations.triggered)
- EmailNotificationService — JavaMailSender, Thymeleaf email template
- DatadogEventService — Datadog Events v1 API
- KafkaNotifierConfig — ConcurrentKafkaListenerContainerFactory + DefaultErrorHandler

cb-mcp-server · port 8086

Implements Model Context Protocol (MCP) SSE transport at /sse. 12 tools for AI assistants (Claude Desktop, Cursor). Engineers can query past fixes, trigger rebuilds, search Qdrant, fetch traces, create PRs, and roll back merges using natural language.

Key Classes / Beans:

- McpServerConfig — Spring AI MCP server bean, SSE transport
- findPreviousFixes, getSonarIssue, getPRDiff — data query tools
- buildProject, createPR, triggerRollback — action tools
- getBuildLog, regenerateFix — pipeline management tools
- searchPastIncidents, queryQdrant — RAG search tools
- getTrace (Tempo), getMetrics (Prometheus) — observability tools

cb-escalation · port 8089

Consumes EscalationKafkaEvent. Generates RCA report via GPT-4o. Creates Jira ticket with RCA attached. Posts Teams adaptive card with severity badge, retry history, and Jira link. All three outputs are feature-flagged (JIRA_ENABLED, TEAMS_ENABLED) for gradual activation.

Key Classes / Beans:

- EscalationConsumer — @KafkaListener(escalations.triggered)
- RcaGenerationService — GPT-4o prompt: vuln + retry history + build logs → RCA markdown
- JiraEscalationService — Jira REST API v3: create issue + attach RCA as file
- TeamsNotificationService — Teams Incoming Webhook: adaptive card JSON payload
- EscalationConfig — ConcurrentKafkaListenerContainerFactory + escalationExecutor

cb-api · port 8080

Public REST API. Spring Security API-key filter (X-API-Key header). SpringDoc OpenAPI 3 + Swagger UI. All read/write on vulnerabilities, fixes, metrics, cost, audit. Prometheus metrics at /actuator/prometheus. Ingress routes compliance-buddy.local/* to this service.

Key Classes / Beans:

- VulnerabilityController, FixController, MetricsController, AuditController
- CostTrackingController — /cost/summary, /cost/by-cwe, /cost/by-model
- ScanController — triggers on-demand SonarQube poll via cb-scanner
- ApiKeyAuthFilter — Spring Security OncePerRequestFilter
- OpenApiConfig — SpringDoc bean with API-key security scheme

7. REST API Contract

■ All endpoints require header `X-API-Key: dev-key-change-in-prod`. Swagger UI: <http://compliance-buddy.local/swagger-ui/index.html> | `/actuator/health` is auth-exempt (k8s probes).

Method	Path	Request	Response	Description
POST	<code>/api/v1/scans/{projectKey}</code>	—	<code>{ newFindings }</code>	Trigger SonarQube poll
GET	<code>/api/v1/vulnerabilities</code>	<code>?status=&severity;=</code>	Page	List vulnerabilities
GET	<code>/api/v1/vulnerabilities/{id}</code>	—	Vulnerability	Single vulnerability
POST	<code>/api/v1/vulnerabilities/retry/{id}</code>	—	<code>{ queued }</code>	Re-queue AI fix
GET	<code>/api/v1/fixes</code>	—	List	All generated fixes
GET	<code>/api/v1/fixes/{id}</code>	—	Fix	Fix with full diff
GET	<code>/api/v1/metrics</code>	—	<code>{ vulns{}, fixes{}, rate }</code>	Dashboard KPIs
GET	<code>/api/v1/cost/summary</code>	<code>?days=7</code>	<code>{ totalCostUsd, tokens }</code>	Cost summary
GET	<code>/api/v1/cost/by-cwe</code>	<code>?days=30</code>	<code>[[cweId, totalCostUsd]]</code>	Cost per CWE
GET	<code>/api/v1/cost/by-model</code>	<code>?days=30</code>	<code>[[model, totalCostUsd]]</code>	Cost per model
GET	<code>/api/v1/audit</code>	—	List	Full audit trail
GET	<code>/api/v1/audit/entity/{id}</code>	—	List	Per-entity audit
GET	<code>/actuator/health</code>	—	<code>{ status: UP }</code>	Health (no auth)
GET	<code>/actuator/prometheus</code>	—	Prometheus text	Metrics scrape endpoint

8. Security Design

API Authentication

- All REST endpoints protected by ApiKeyAuthFilter (OncePerRequestFilter).
- Multiple keys via CB_API_KEYS comma-separated list — rotate without downtime.
- /actuator/health exempt for k8s liveness/readiness probes.

Secrets Management

- All credentials in Kubernetes Secrets (k8s/base/secrets.yaml), not env vars at build time.
- HashiCorp Vault integration available (spring.cloud.vault.enabled=true, off by default).
- No credentials embedded in Docker images.

Kafka Security

- ErrorHandlerDeserializer prevents deserialization attacks from malformed payloads.
- spring.json.use.type.headers=false blocks __TypeId__ header injection.
- spring.json.value.default.type pins deserialization class — no dynamic class loading.

OPA Governance Gate

- Rego policies evaluated server-side — policy changes need no code deployment.
- cb-pr hard-fails (escalates) on OPA timeout or connection error — fail-closed.
- All policy decisions logged to audit_events for compliance reporting.

Container Security

- All CB images run as non-root user cbapp (UID 1001).
- Base image: eclipse-temurin:17-jre-jammy — minimal JRE, no JDK, no build tools.
- -XX:+UseContainerSupport ensures JVM respects cgroup memory limits.

Network Isolation

- All inter-service communication within cb-system namespace (ClusterIP only).
- Only cb-api exposed via Ingress. MongoDB, Redis, OPA are not externally accessible.
- Qdrant and Elasticsearch bound to cluster-internal ClusterIP services.

9. Observability

9.1 Prometheus Metrics

Metric	Type	Labels	Description
cb_vulnerabilities_detected_total	Counter	projectKey, severity, cweld	Vulns ingested
cb_fixes_generated_total	Counter	model, cacheHit, cweld	AI fixes produced
cb_prs_created_total	Counter	projectKey, severity	GitHub PRs opened
cb_escalations_triggered_total	Counter	reason	Jira/Teams escalations
cb_llm_cost_usd_total	Counter	model, cweld	Cumulative LLM spend
cb_llm_tokens_total	Counter	model, direction	Input+output tokens
cb_fix_confidence	Gauge	model, cweld	Last fix confidence (0–1)
cb_pipeline_duration_seconds	Histogram	severity	Detection→PR latency
cb_kafka_consumer_lag	Gauge	topic, group	Consumer lag per topic
cb_redis_cache_hit_total	Counter	cweld	Prompt cache hits
cb_build_validation_duration_seconds	Histogram	buildTool	Gradle/Maven build time

9.2 Distributed Tracing (Tempo)

All services export OTLP traces to Tempo at `http://tempo:4318/v1/traces` (100% sampling). Every vulnerability processing message carries `vulnerabilityId` as a baggage attribute for end-to-end correlation from `cb-scanner` through `cb-pr`. Query in Grafana Explore → Tempo by `traceId` or attribute.

9.3 Log Aggregation (Loki + Promtail)

Promtail DaemonSet tails all pod logs in `cb-system`, labelling with `app`, `namespace`, `pod`, `container`. All CB services emit structured JSON logs including `vulnerabilityId`, `fixId`, `model`, `cacheHit`, and `duration`. Query in Grafana Explore → Loki data source.

10. CWE Coverage

CWE	Name	Fix Strategy	Confidence
CWE-89	SQL Injection	PreparedStatement / JPA named parameters	0.93
CWE-79	XSS	HtmlUtils.htmlEscape + CSP header injection	0.91
CWE-78	OS Command Injection	ProcessBuilder(List) replacing Runtime.exec(String)	0.95
CWE-22	Path Traversal	Path.toRealPath() + base-dir prefix assertion	0.90
CWE-798	Hardcoded Credentials	Extract to @Value + Kubernetes Secret	0.96
CWE-327	Broken Crypto	MD5/SHA-1→SHA-256; DES→AES-256-GCM	0.92
CWE-918	SSRF	URL allow-list validation before any HTTP client call	0.88
CWE-330	Weak Randomness	Math.random()/new Random() → SecureRandom	0.97
CWE-502	Unsafe Deserialization	Replace Java deserialization with Jackson JSON	0.89
CWE-611	XXE	Disable DOCTYPE + external entities on DocumentBuilderFactory	0.94
CWE-200	Information Exposure	Remove stack traces from HTTP; add @ExceptionHandler	0.85
CWE-352	CSRF	Spring Security CSRF token on state-changing endpoints	0.91

11. Deployment & Operations

11.1 Local Development (k3d)

Step	Command	Notes
1. Start cluster	start-cb.ps1	Docker + k3d + pod readiness wait (8-min timeout)
2. Apply secrets	kubectl apply -k k8s/base/	Edit secrets.yaml before first run
3. Build module	./gradlew :cb-agent:bootJar	Requires Java 17. Use Bash, not PowerShell.
4. Build image	docker build -f cb-agent/Dockerfile.prebuilt -t cb-agent:latest cb-agent/	Build context = module dir (not repo root — .dockerignore excludes **/build at root)
5. Load to k3d	k3d image import cb-agent:latest -c compliance-buddy	Run from Bash
6. Restart pod	kubectl rollout restart deployment/cb-agent -n cb-system	
7. Watch logs	kubectl logs -n cb-system -l app=cb-agent -f	
8. Status check	status-cb.ps1	Shows pods, services, URLs, known issues
9. Stop cluster	stop-cb.ps1	All data preserved (PVCs remain)

11.2 Dockerfile.prebuilt Pattern

All CB modules use identical Dockerfile.prebuilt. The root .dockerignore excludes **/build so the build context must be the module directory, not the repository root.

```
FROM eclipse-temurin:17-jre-jammy
RUN groupadd -r cbapp && useradd -r -g cbapp -u 1001 cbapp
WORKDIR /app
COPY build/libs/cb--1.0.0.jar app.jar
RUN chown cbapp:cbapp app.jar
USER cbapp
EXPOSE
ENTRYPOINT ["java", "-XX:+UseContainerSupport", "-XX:MaxRAMPercentage=75.0",
"-Djava.security.egd=file:/dev/./urandom", "-jar", "app.jar"]
```


12. Known Issues & Limitations

Issue	Pod / Component	Impact	Workaround / Fix
cb-python-embedder ImagePullBackOff	cb-python-embedder (:8090)	Python FastAPI embedding service not running. Falls back to Spring AI inline text-embedding-3-small via OpenAI HTTP.	Build Python FastAPI image and push to local k3d registry. Or keep Spring AI inline embedding (same model, negligible perf difference).
OPA ImagePullBackOff	opa (:8181)	Governance gate disabled. cb-pr runs with OPA_ENABLED=false — all AI fixes bypass policy check.	Pull OPA 0.68 image via docker pull, then k3d image import. P0 fix — every PR should pass policy.
Ollama CrashLoopBackOff	ollama (:11434)	Local LLM inference unavailable. MAJOR/MINOR/INFO vulns route to GPT-4o instead of free local models — increases LLM spend ~60%.	Add GPU resource requests to Ollama Deployment or increase node RAM limits. P0 fix for cost efficiency.
emptyDir /workspace/repo	cb-patcher	Repository clone lost on pod restart. First request after restart re-clones the full repo (adds ~10–30s latency).	Expected behaviour — intentional to avoid stale checkout state. Future: persistent PVC workspace with git fetch instead of full clone.
GitHub Copilot reviewer failure	cb-pr	If GITHUB_REVIEWER_USERNAME is a GitHub Copilot bot account, it may not be a repo collaborator and the reviewer request 422s.	Set GITHUB_REVIEWER_USERNAME to an actual human GitHub username or a team slug (@org/team-slug).
Version1 / VersionOne API	cb-pr	VersionOne work item creation is optional. If VERSION1_API_URL is blank, cb-pr skips ticket creation silently.	Set VERSION1_API_URL and VERSION1_TOKEN or switch to Jira-only via cb-escalation.

13. Future Enhancements

P0 — Critical Path

OPA image deployment

Build OPA 0.68, push to k3d registry — activates governance gate for all PRs.

Ollama GPU scheduling

GPU resource requests on Ollama Deployment — restores free local LLM inference.

Multi-repo support

Database-backed project registry with per-project credentials and polling intervals.

P1 — High Value

PR auto-merge on CI pass

GitHub Actions webhook consumer in cb-pr — fully autonomous zero-touch pipeline.

cb-python-embedder activation

Deploy FastAPI with text-embedding-3-large (3072-d) for +8–12% retrieval accuracy.

Cost budget alerts

MAX_DAILY_COST_USD threshold: when exceeded, force all models to Ollama until midnight UTC.

Slack notifications

SLACK_ENABLED toggle in cb-notifier alongside existing Teams support.

Fix confidence trend dashboard

Grafana panel: cb_fix_confidence over time per model and CWE.

P2 — Platform Scale

Horizontal pod autoscaling

HPA for cb-agent (CPU 70%) + cb-patcher (CPU 60%) — linear throughput scaling.

Dead letter queue UI

cb-dlq consumer stores DLT messages in MongoDB; Swagger manual replay endpoint.

Kafka Schema Registry

Confluent Schema Registry + AvroDeserializer — enforced forward/backward compatibility.

3-broker KRaft cluster

Production HA: replication factor 3, min-ISR 2. Required for SLA > 99.5%.

GitLab + Bitbucket PR support

Abstract GitHostClient interface; GitLabPrService + BitbucketPrService implementations.

P3 — AI Research

Fine-tuned CWE models

LoRA adapters on qwen2:7b per CWE — outperforms GPT-4o on high-freq CWEs at 1/100th cost.

Parallel LangGraph4j retrieval

Qdrant and Elasticsearch retriever nodes run in parallel — -40% p99 latency.

Agentic code review

Sub-graph in reviewer node: security review, perf review, style review → majority vote.

Vulnerability prediction

Classifier on historical fix data: predict CWE-89/79/78 risk before SonarQube runs.

RLHF / DPO fine-tuning

Capture reviewer edits as preference signal → quarterly DPO fine-tune of generator model.

P4 — Enterprise Readiness

LDAP / SSO for cb-api

Spring Security OAuth2 Resource Server (JWT) — Entra ID, Okta, Keycloak support.

Multi-tenancy

Partition MongoDB + Kafka topics by tenantId — required for SaaS offering.

Compliance report generation

/api/v1/reports/compliance → PDF/Excel of detected/remediated/escalated vulns per CWE.

Helm chart packaging

One-command install: helm install compliance-buddy oci://ghcr.io/org/cb-helm/compliance-buddy.

14. Technology Stack

Category	Technology	Version	Role
Language	Java	17 LTS	All CB services
Framework	Spring Boot	3.3.6	Application scaffold + auto-configuration
AI Orchestration	LangGraph4j	1.5.14	Multi-agent StateGraph in cb-agent
AI Integration	Spring AI	1.0.0	ChatClient, VectorStore, MCP SDK
LLM Primary	OpenAI GPT-4o	gpt-4o	CRITICAL/BLOCKER fix generation + RCA
LLM Review	Anthropic Claude	claude-sonnet-4-6	Independent code review + fallback
LLM Local	Ollama (llama3:8b, qwen2:7b)	0.3	MAJOR/MINOR/INFO — zero API cost
Event Streaming	Apache Kafka	KRaft 3.7	Async pipeline bus — no ZooKeeper
Spring Kafka	Spring Kafka	3.2.5	Producer/consumer + ErrorHandlingDeserializer
Database	MongoDB	7.0	Primary datastore (all 6 collections)
Cache	Redis	7.2	LLM prompt cache (Lettuce, 24 h TTL)
Vector Store	Qdrant	1.9.2	Semantic RAG — text-embedding-3-small 1536-d
Lexical Search	Elasticsearch	8.13	BM25 RAG index (hybrid with Qdrant)
Policy Engine	OPA	0.68	Rego PR governance gate
Git Operations	JGit	6.8	Diff application + push in cb-patcher
API Docs	SpringDoc OpenAPI	2.5	Swagger UI on cb-api
Resilience	Resilience4j	2.2	CircuitBreaker on SonarQube + GitHub calls
Metrics	Micrometer + Prometheus	1.13 / 2.52	Application metrics + scraper
Tracing	OpenTelemetry + Tempo	1.39 / 2.5	Distributed traces (OTLP HTTP)
Logging	Logback → Loki	—	Structured JSON logs via Promtail DaemonSet
Dashboards	Grafana	11.1	Pipeline + JVM + Kafka dashboards
Containers	Docker + k3d + k3s	k3d 5.8	Local Kubernetes cluster
Build	Gradle	8.14	Multi-module build, JVM pinned to JDK 17
Protocol	MCP SSE	1.0	cb-mcp-server AI tool interface

Appendix — Configuration Reference

Environment Variable	Default	Module(s)	Description
CB_API_KEYS	dev-key-change-in-prod	cb-api	Comma-sep valid REST API keys
OPENAI_API_KEY	—	cb-agent, cb-escalation	GPT-4o API key
ANTHROPIC_API_KEY	—	cb-agent	Claude reviewer + fallback
SONARQUBE_URL	http://cb-sonarqube:9000/sonar	cb-scanner	SonarQube instance
SONARQUBE_TOKEN	—	cb-scanner	SonarQube user token
SCANNER_PROJECTS	—	cb-scanner	Comma-sep SonarQube project keys
GITHUB_TOKEN	—	cb-pr, cb-mcp-server	GitHub PAT (repo scope)
GITHUB_OWNER	—	cb-pr	GitHub org or username
GITHUB_REPO	—	cb-pr	Repository name
GITHUB_REVIEWER_USERNAME	—	cb-pr	Auto-requested PR reviewer
VERSION1_API_URL	—	cb-pr	VersionOne instance URL (optional)
VERSION1_TOKEN	—	cb-pr	VersionOne API token
JIRA_ENABLED	false	cb-escalation	Enable Jira ticket creation
JIRA_BASE_URL	—	cb-escalation	https://org.atlassian.net
JIRA_EMAIL	—	cb-escalation	Jira account email
JIRA_API_TOKEN	—	cb-escalation	Jira API token
JIRA_PROJECT_KEY	CB	cb-escalation	Jira project key for escalations
TEAMS_ENABLED	false	cb-escalation	Enable Teams adaptive card
TEAMS_WEBHOOK_URL	—	cb-escalation	Teams Incoming Webhook URL
SMTP_HOST	smtp.gmail.com	cb-notifier	SMTP server
SMTP_PORT	587	cb-notifier	SMTP port
SMTP_USERNAME	—	cb-notifier	SMTP credentials
SMTP_PASSWORD	—	cb-notifier	SMTP password / app password

NOTIFIER_TEAM_EMAILS	—	cb-notifier	Comma-sep recipient emails
DATADOG_API_KEY	none	cb-notifier	Datadog Events API key
OPA_ENABLED	true	cb-pr	OPA governance gate toggle
KAFKA_BOOTSTRAP_SERVERS	kafka:9092	all	Kafka broker
MONGODB_URI	mongodb://cb-mongodb:27017/compliance_buddy	all	MongoDB URI
VAULT_ENABLED	false	all	HashiCorp Vault integration
OTEL_ENABLED	true	all	OpenTelemetry tracing
OTEL_EXPORTER_OTLP_ENDPOINT	http://tempo:4318/v1/traces	all	Tempo OTLP endpoint